TP2 & TD2 : ColorImage, format PPM et rééchantillonnage

R3.L01 — Représentations et codages des images

Fiche d'explication du TP, écrite à partir du cours (CM diapos 29 à 39), du TD2 (exercice 1), du corrigé de rééchantillonnage pris en TD et du sujet du TP2.

Code complet et testé : `PROJET_FINAL/Image.hpp`, `Image.cpp`, `tp2.cpp` — l'exercice 2 et 3 du TD2 (endianness, tailles) est dans la fiche de révision.

Ce qu'il faut rendre

Dans les MÊMES fichiers Image.hpp et Image.cpp que GrayImage (sujet du TP2)

```
class Color { public: uint8_t r, g, b; ... };      // + ==,
double * Color, Color + Color
class ColorImage {
  ...                                               // même
base que GrayImage
  void clear(Color color = Color());                // noir
par défaut
  void rectangle    (x, y, w, h, Color color);
  void fillRectangle(x, y, w, h, Color color);
  void writePPM(std::ostream&) const;
  static ColorImage* readPPM(std::istream&);
  ColorImage* simpleScale  (uint16_t w, uint16_t h) const;
  ColorImage* bilinearScale(uint16_t w, uint16_t h) const;
};
```

Le TD3 corrigé en classe fait `simpleScale` sur **GrayImage** : notre code fournit donc les deux rééchantillonnages pour les deux classes.

Tests demandés par le sujet

1. Lire `chat.ppm`, tracer des rectangles colorés, écrire `tp2_chat.ppm`.
2. Lire `chat_petit.ppm` (80 × 60), l'agrandir en 823 × 400 : `tp2_simple.ppm` puis `tp2_bilinear.ppm`.
3. Bonus : lire un PPM « P3 » (texte).

```
cd PROJET_FINAL
make tp2 && ./tp2
eog tp2_simple.ppm tp2_bilinear.ppm
```

La classe Color et ses opérateurs CM diapo 38

```
class Color {
public:
  uint8_t r, g, b;   // Gardez « public » ces trois données.
  inline Color(uint8_t _r = 0, uint8_t _g = 0, uint8_t _b = 0)
    : r(_r), g(_g), b(_b) {}
  friend bool  operator==(const Color& c1, const Color& c2);
  friend Color operator* (double alpha, const Color& color);
  friend Color operator+ (const Color& c1, const Color& c2);
};

// Ramène un réel dans [0;255] puis l'arrondit à l'entier le plus
proche.
static uint8_t to_byte(double v)
{
  if (v <= 0)   return 0;
  if (v >= 255) return 255;
  return uint8_t(v + 0.5);
}
Color operator*(double alpha, const Color& color)
{ return Color(to_byte(alpha * color.r), to_byte(alpha * color.g),
to_byte(alpha * color.b)); }
Color operator+(const Color& c1, const Color& c2)   // addition «
saturée »
{ return Color(to_byte(c1.r + c2.r), to_byte(c1.g + c2.g),
to_byte(c1.b + c2.b)); }
```

- **Pourquoi ces opérateurs ?** Pour écrire la formule bilinéaire *exactement* comme en niveaux de gris : `(1-a)*c1 + a*c2`, avec des `Color`.
- `friend` : l'opérateur n'est pas une méthode (le `double` est à gauche du `*`), mais on le déclare dans la classe.
- `==` est indispensable pour la correction automatique (CM).

Pourquoi saturer et arrondir ?

`uint8_t` va de 0 à 255 : 200 + 100 = 300 deviendrait **44** (300 − 256). On bloque à 255.

0,5 × 255 = 127,5 : tronqué, ça donne 127 et l'image s'assombrit un peu à chaque calcul ; arrondi, ça donne 128.

ColorImage : le même code que GrayImage CM diapo 39

```
class ColorImage {          // Remplacer Gray par Color
  const uint16_t width, height;
        Color    *array;   // Remplacer uint8_t par
Color
  ColorImage() = delete;
public:
  ColorImage(uint16_t w, uint16_t h);
  ColorImage(const ColorImage& orig);
  ~ColorImage();
  inline Color& pixel(uint16_t x, uint16_t y) { ... }
  ...
};
```

Le CM le dit : dupliquer le code est mauvais pour la maintenance. Une classe modèle `Image<T>` serait plus propre, mais le prof demande de ne la tenter que si on est à l'aise en C++. On garde donc la duplication, plus simple à relire.

En mémoire, un tableau de `Color` est une suite d'octets r, g, b, r, g, b... soit **exactement** l'ordre des pixels d'un fichier PPM :



C'est ce qui permet d'écrire tous les pixels d'un seul `write`. Mais seulement si une `Color` fait 3 octets, sans octet de remplissage (*padding*, CM diapo 14). C'est le cas ici : trois `uint8_t`, et une classe est alignée sur sa donnée la plus exigeante, soit 1 octet. Donc `sizeof(Color)` vaut 3.

Le format PPM « P6 » : l'exemple du TD2, octet par octet

```
00000000  50 36 0a 23 20 4d 61 64 65 20 62 79 20 45 2e 20
00000010  52 65 6d 79 0a 33 20 33 0a 32 35 35 0a 00 00 00
00000020  80 80 80 ff ff ff ff 00 00 00 ff 00 00 00 ff 00
00000030  ff ff ff 00 ff ff ff 00
```

"P6" + LF
"# Made by E. Remy" + LF
"3 3" + LF
"255" + LF
3 × 3 × 3 = 27 octets

29 octets d'en-tête + 27 octets de pixels = 56 = 0x38 octets.

	x=0	**x=1**	**x=2**
y=0	`00 00 00` noir	`80 80 80` gris	`ff ff ff` blanc
y=1	`ff 00 00` rouge	`00 ff 00` vert	`00 00 ff` bleu
y=2	`00 ff ff` cyan	`ff 00 ff` magenta	`ff ff 00` jaune

- Même structure que PGM, avec **"P6"** au lieu de "P5".
- **3 octets par pixel**, dans l'ordre **rouge, vert, bleu**.
- Ligne 3 : cyan = vert + bleu, etc. : les couleurs secondaires de RGB sont les primaires de CMY (CM diapo 31).

writePPM et readPPM TD2 exercice 1

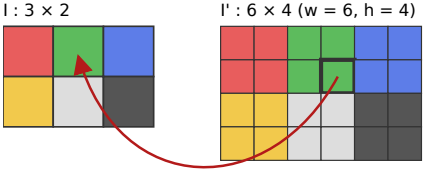
```
void ColorImage::writePPM(std::ostream& os) const
{
  os << "P6\n"
     << "# Image sauvegardée par " << AUTEUR
     << " pour les TP de RepCod.\n"
     << width << " " << height << "\n"
     << 255 << "\n";
  // 3 octets par pixel, dans l'ordre r, g, b :
  // c'est exactement l'ordre en mémoire.
  os.write((const char*)array, size_t(width) * height *
3);
  if (!os)
    throw std::runtime_error("writePPM : erreur
d'écriture");
}
```

`skip_line` et `skip_comments` sont des fonctions libres : PGM et PPM les partagent (`chat.ppm` a lui aussi des commentaires après les dimensions).

`readPPM` est **la copie de** `readPGM` avec trois différences :

```
if (magic != "P6" && magic != "P3")      // 1. magic number
  throw std::runtime_error("readPPM : ...");
...
ColorImage* res = new ColorImage(uint16_t(w), uint16_t(h));
if (magic == "P6") {
  is.get();                              // un seul blanc
après 255
  is.read((char*)res->array, size_t(w) * h * 3);  // 2. × 3
} else {                                 // 3. "P3" : r g b
en texte
  for (size_t t = 0; t < size_t(w) * h; t++) {
    unsigned int r = 0, g = 0, b = 0;
    if (!(is >> r >> g >> b) || r > 255 || g > 255 || b >
255) {
      delete res; throw std::runtime_error("...");
    }
    res->array[t] = Color(uint8_t(r), uint8_t(g),
uint8_t(b));
  }
}
if (!is) { delete res; throw std::runtime_error("..."); }
```

Rééchantillonnage « simple » : le plus proche voisin TD3



(x', y') = (3, 1) → x = 3 × 3 / 6 = 1, y = 1 × 2 / 4 = 0

Le principe

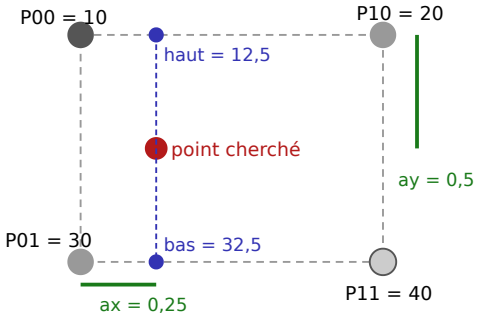
On parcourt **l'image d'arrivée** I' et, pour chaque pixel (x', y'), on va chercher le pixel de départ correspondant :

x = x' × width / w **y = y' × height / h** (division entière)

```
// Corrigé du TD3 (GrayImage ; idem pour ColorImage)
GrayImage* GrayImage::simpleScale(uint16_t w, uint16_t h)
const
{
  GrayImage* res = new GrayImage(w, h);
  for (uint16_t y_prime = 0; y_prime < h; y_prime++) {
    uint16_t y = uint16_t(uint32_t(y_prime) * height / h);
    for (uint16_t x_prime = 0; x_prime < w; x_prime++) {
      uint16_t x = uint16_t(uint32_t(x_prime) * width / w);
      res->pixel(x_prime, y_prime) = pixel(x, y);
    }
  }
  return res;
}
```

- **Pourquoi parcourir I' et pas I ?** En parcourant I, on n'écrirait qu'un pixel de I' sur 2 (ou sur 10) : I' aurait des trous. En parcourant I', **chaque** pixel reçoit une valeur.
- `x' < w` donc `x < width` ; `uint32_t` car `x' × width` peut dépasser 65535.
- Résultat : chaque pixel devient un **bloc uniforme** (effet « gros pixels »).

Rééchantillonnage bilinéaire : mélanger les 4 voisins



Le point cherché tombe en général **entre** 4 pixels. Au lieu de prendre le plus proche, on mélange les 4, avec un poids d'autant plus grand que le pixel est proche :

La formule (deux interpolations linéaires)

haut = (1 − ax) × P00 + ax × P10
bas = (1 − ax) × P01 + ax × P11
valeur = (1 − ay) × haut + ay × bas

Exemple (le schéma)

haut = 0,75 × 10 + 0,25 × 20 = 12,5
bas = 0,75 × 30 + 0,25 × 40 = 32,5
valeur = 0,5 × 12,5 + 0,5 × 32,5 = **22,5 → 23**

ax et ay sont les parties fractionnaires de la position réelle (x, y) : si x = 7,25 alors x0 = 7, x1 = 8 et ax = 0,25. Le bord droit (x0 = width − 1) n'a pas de voisin : on prend x1 = x0.

Le décalage d'un demi-pixel (remarque du sujet)



x' =	0	1	2	3	4	5	6	7
sans correction	0	0,5	1	1,5	2	2,5	3	3,5
avec correction	−0,25	0,25	0,75	1,25	1,75	2,25	2,75	3,25

sans : $x = x' \times 4 / 8$; avec : $x = (x' + 0{,}5) \times 4 / 8 - 0{,}5$. Le centre du pixel 0 de I' est au quart du pixel 0 de I.

Le problème : un pixel est un carré, pas un point. `x = x' × W / w` fait correspondre les **bords gauches** des pixels : en bilinéaire, toutes les positions sont décalées vers la gauche et le haut.

Agrandi ≈ 10 fois (823 / 80), un demi-pixel source devient **≈ 5 pixels** : on le voit en faisant alterner les deux images. Sans correction, on a quand même « pratiquement tous les points » (sujet).

```
// -0,5 / +0,5 avant et après le changement de repère
static void bilinear_coord(uint16_t p, uint16_t dst,
uint16_t src,
                           uint16_t& p0, uint16_t& p1,
double& a)
{
  double s = (p + 0.5) * src / dst - 0.5;
  if (s < 0)       s = 0;        // bords : on reste
dans l'image
  if (s > src - 1) s = src - 1;
  p0 = uint16_t(s);
  p1 = (p0 + 1 < src) ? uint16_t(p0 + 1) : p0;
  a  = s - p0;                   // dans [0;1[
}
```

bilinearScale : le code

```
ColorImage* ColorImage::bilinearScale(uint16_t w,
uint16_t h) const
{
  ColorImage* res = new ColorImage(w, h);
  for (uint16_t y_prime = 0; y_prime < h; y_prime++) {
    uint16_t y0, y1; double ay;
    bilinear_coord(y_prime, h, height, y0, y1, ay);
    for (uint16_t x_prime = 0; x_prime < w; x_prime++)
{
      uint16_t x0, x1; double ax;
      bilinear_coord(x_prime, w, width, x0, x1, ax);
      Color haut = (1 - ax) * pixel(x0, y0) + ax *
pixel(x1, y0);
      Color bas  = (1 - ax) * pixel(x0, y1) + ax *
pixel(x1, y1);
      res->pixel(x_prime, y_prime) = (1 - ay) * haut +
ay * bas;
    }
  }
  return res;
}
```

Pour `GrayImage`, c'est la même chose avec des `double`, et on arrondit à la fin : `uint8_t(v + 0.5)`.

Grâce aux opérateurs de `Color`, la formule est **identique** à celle des niveaux de gris : c'est exactement ce que demande le sujet (« pour ne pas avoir à changer la formule vue en TD »).

Ce que le bilinéaire ne fait pas

Il n'ajoute **aucune information** : l'image 823 × 400 contient autant de détails que l'image 80 × 60. Il remplace les marches d'escalier par un dégradé, donc l'image est plus agréable mais floue.

Remarquez aussi l'aspect « aplati » : 823 / 400 ≠ 80 / 60, le **rapport d'aspect** change.

Résultats : 80 × 60 agrandi en 823 × 400



simpleScale : `tp2_simple.ppm`



bilinearScale : `tp2_bilinear.ppm`



zoom : blocs uniformes



zoom : dégradés entre les pixels d'origine

Ce qui a été corrigé par rapport à la version de Gemini

Point	Version Gemini	Correction
Fichiers	`ColorImage.hpp/.cpp`, et **GrayImage a disparu** du TP2 (chaque dossier de TP avait sa propre copie, différente).	Un seul `Image.hpp/.cpp` avec GrayImage, Color et ColorImage, comme le demande le sujet.
`Color + Color`	200 + 100 donnait 44 (débordement).	Addition saturée à 255.
`double * Color`	Troncature (l'image s'assombrit).	Arrondi au plus proche.
Rééchantillonnage	Seulement pour ColorImage ; pas de correction du demi-pixel.	Les deux classes (le corrigé du TD est sur GrayImage) ; correction ±0,5.
`writePPM`	Écrit le tableau en bloc sans justifier la taille d'une `Color`.	Justifié par le CM diapo 14 : trois `uint8_t`, pas de remplissage.
Commentaire PPM	Sans nom.	« Image sauvegardée par *nom* ... » (TD2 question 2).

Questions possibles au contrôle

1. **Taille des pixels d'un PPM 320 × 240 ?** 320 × 240 × 3 = 230 400 octets (+ l'en-tête).
2. **Différences PGM / PPM ?** "P6" au lieu de "P5", 3 octets (r, g, b) par pixel au lieu de 1.
3. **Pourquoi parcourir l'image d'arrivée ?** Pour donner une valeur à chacun de ses pixels (sinon des trous en agrandissant).
4. **simpleScale de 80 à 823 colonnes : quel x pour x' = 500 ?** 500 × 80 / 823 = 48 (division entière).
5. **Calculer un pixel bilinéaire** à partir de 4 voisins et de (ax, ay) : voir l'exemple 10, 20, 30, 40 → 22,5.
6. **Pourquoi des opérateurs dans Color ?** Pour garder la formule du TD sans la réécrire composante par composante.
7. **Pourquoi une image floue ?** Le bilinéaire fait des dégradés entre pixels d'origine ; il n'invente pas de détails.
8. **Pourquoi l'image paraît aplatie ?** Le rapport largeur / hauteur n'est pas conservé (823/400 ≈ 2,06 contre 80/60 ≈ 1,33).